

UNIDAD 1

Fundamentos de la Programación Orientada a Objetos

Tema:



UEA
UNIVERSIDAD
ESTATAL AMAZÓNICA

**CARACTERÍSTICAS DE LA
PROGRAMACIÓN ORIENTADA A
OBJETOS**

GUÍA DE ESTUDIO

SEMANA 4

ÍNDICE

1.1. INTRODUCCIÓN.....	1
1.2. RESULTADO DE APRENDIZAJE	2
1.3. RUTA DE APRENDIZAJE	2
1.4. DESARROLLO DE CONTENIDOS.....	3
1.4.1 Características principales de la POO.....	3
1.4.2 La clase como modelo de representación	4
1.4.3 Constructor, self y atributos.....	6
1.4.4 Métodos y comportamientos de los objetos.....	7
1.4.5 Método especial __str__().....	8
1.4.6 Organización modular del proyecto.....	9
1.4.7 Modelos: representación de entidades del sistema	10
1.4.8 Servicios: lógica de operación y administración de objetos	12
1.4.9 Main.py: punto de arranque de la aplicación	13
1.4.10 Relación entre objetos.....	15
1.4.11 Uso de importaciones en Python.....	16
1.4.12 Flujo general de ejecución del proyecto	17
1.4.13 Beneficios de la organización modular.....	18
1.5. CONCLUSIONES.....	19
1.6. BIBLIOGRAFÍA.....	20

1.1. INTRODUCCIÓN

La Programación Orientada a Objetos (POO) es un paradigma de programación que permite organizar el software a partir de entidades llamadas objetos. Cada objeto puede almacenar información mediante atributos y ejecutar acciones mediante métodos. Esta forma de trabajo facilita que el programador represente elementos del mundo real dentro de un sistema computacional, manteniendo una relación clara entre los datos y los comportamientos que los utilizan.

En las semanas anteriores se revisaron los fundamentos iniciales de la POO, la diferencia entre programación tradicional y programación orientada a objetos, así como la importancia de organizar el código de acuerdo con el tipo de problema que se desea resolver. A partir de esta base, la Semana 4 profundiza en las características de la Programación Orientada a Objetos, pero no solo desde una perspectiva teórica, sino mediante el análisis de un proyecto Python organizado en varios archivos.

El tema oficial de esta semana corresponde a las características de la Programación Orientada a Objetos. Por ello, esta guía conserva dicho enfoque, pero lo desarrolla mediante una aplicación práctica. En lugar de estudiar únicamente definiciones aisladas, se analizará cómo una clase se convierte en un modelo, cómo los objetos se crean a partir de ella, cómo los atributos almacenan información, cómo los métodos permiten ejecutar comportamientos y cómo la modularidad contribuye a organizar un programa de manera más clara.

El caso de estudio utilizado será un sistema básico de biblioteca. Este proyecto no incluye todavía menús, bases de datos, interfaces gráficas ni manejo avanzado de archivos, ya que el propósito de la semana es comprender la estructura interna de un programa orientado a objetos. El sistema se organiza en carpetas como modelos, servicios y un archivo principal `main.py`. Esta estructura permitirá introducir progresivamente la separación de

responsabilidades y preparar al estudiante para desarrollar aplicaciones más completas en semanas posteriores.

Comprender esta organización es fundamental para avanzar hacia proyectos de mayor complejidad. Cuando todo el código se escribe en un solo archivo, puede resultar sencillo al inicio, pero difícil de mantener cuando el programa crece. En cambio, cuando el software se divide en módulos, cada parte cumple una función específica. Esta forma de organización mejora la lectura, facilita la reutilización y permite que el sistema pueda ampliarse sin perder claridad.

La presente guía tiene como propósito ofrecer al estudiante un material de estudio amplio, ordenado y fundamentado. Cada apartado explica un concepto clave de la POO y lo relaciona con el código del proyecto biblioteca_app. De esta manera, el estudiante no solo memoriza definiciones, sino que aprende a reconocer cómo dichas características aparecen dentro de una aplicación escrita en Python.

1.2. RESULTADO DE APRENDIZAJE

Interpreta los conceptos fundamentales de la Programación Orientada a Objetos para el desarrollo de soluciones computacionales de complejidad media.

1.3. RUTA DE APRENDIZAJE

La evaluación del aprendizaje se desarrollará mediante actividades correspondientes a los componentes en contacto con el docente, práctico-experimental y autónomo. Estas actividades están orientadas a fortalecer la comprensión de las características de la POO mediante el análisis de código Python y la organización modular de un proyecto sencillo.

COMPONENTE	ACTIVIDAD	CALIFICACIÓN
Componente en contacto con el docente	Cuestionario sobre las características de la Programación Orientada a Objetos.	10 puntos

COMPONENTE	ACTIVIDAD	CALIFICACIÓN
Componente práctico–experimental	Taller práctico de organización modular de un proyecto Python orientado a objetos.	10 puntos
Componente autónomo	Actividad interactiva de refuerzo sobre clases, objetos, atributos, métodos, modularidad e importaciones.	10 puntos

Las actividades permitirán al estudiante observar cómo las características de la POO se reflejan en un proyecto realista y sencillo. El objetivo no es construir todavía una aplicación final completa, sino comprender la función de cada archivo, la relación entre las clases y el flujo de ejecución del programa

1.4. DESARROLLO DE CONTENIDOS

1.4.1 Características principales de la POO

La Programación Orientada a Objetos se basa en organizar el software alrededor de objetos. Un objeto representa una entidad del problema que posee información y comportamientos. Para que esta representación sea posible, la POO utiliza varios conceptos fundamentales: clases, objetos, atributos, métodos y modularidad.

La clase funciona como una plantilla o modelo. Define qué datos tendrá un objeto y qué acciones podrá realizar. El objeto es una instancia concreta creada a partir de una clase. Por ejemplo, la clase Libro puede definir que todo libro tendrá un título, un autor y un ISBN. Sin embargo, cada libro específico tendrá valores propios para esos atributos.

Los atributos almacenan información. Permiten describir el estado de un objeto en un momento determinado. En el caso de un libro, los atributos pueden ser título, autor e isbn. Los métodos, por su parte, representan comportamientos. Permiten que el objeto realice acciones o devuelva información relacionada con sus propios datos.

La modularidad complementa estos conceptos porque permite dividir el programa en partes independientes. En lugar de escribir todas las clases y operaciones en un solo archivo, el proyecto se organiza en carpetas y módulos. Esto mejora la claridad del código y prepara al estudiante para trabajar con aplicaciones de mayor tamaño.

En el proyecto de esta semana, las características de la POO se aplican dentro de una estructura llamada biblioteca_app. En ella se separan los modelos, los servicios y el archivo main.py. Esta organización permite observar cómo los conceptos teóricos se transforman en una estructura práctica de programación.

Característica	Descripción	Aplicación en el proyecto
Clase	Plantilla que define atributos y métodos.	Libro, Usuario y Biblioteca.
Objeto	Instancia creada a partir de una clase.	libro1, libro2, usuario1 y biblioteca.
Atributo	Dato propio de un objeto.	titulo, autor, isbn, nombre, correo.
Método	Acción o comportamiento de un objeto.	mostrar_informacion(), agregar_libro(), mostrar_libros().
Modularidad	Organización del proyecto en partes independientes.	modelos, servicios y main.py.

Nota. Las características de la POO se comprenden mejor cuando se observan funcionando dentro de un proyecto, no únicamente como definiciones aisladas.

1.4.2 La clase como modelo de representación

Una clase es una estructura que permite representar una entidad del problema. En términos sencillos, una clase define cómo serán los objetos que se crearán a partir de ella. Por esta razón se dice que la clase funciona como un molde, plantilla o modelo.

En el proyecto biblioteca_app, la clase Libro representa una entidad fundamental del sistema. Un libro posee información propia, como el título, el autor y el ISBN. Además, puede tener comportamientos asociados, como mostrar su información o presentarse en formato de texto.

El archivo libro.py se ubica dentro de la carpeta modelos porque representa una entidad del sistema. Esto es importante: el modelo no se encarga de ejecutar todo el programa ni de administrar listas completas de libros. Su responsabilidad es representar un libro individual.

```
# modelos/libro.py

class Libro:

    def __init__(self, titulo, autor, isbn):
        self.titulo = titulo
        self.autor = autor
        self.isbn = isbn

    def mostrar_informacion(self):
        return f"Título: {self.titulo} | Autor: {self.autor}"

    def __str__(self):
        return f"{self.titulo} - {self.autor}"
```

La línea `class Libro:` indica que se está definiendo una clase llamada Libro. En Python, por convención, los nombres de las clases suelen escribirse utilizando PascalCase, es decir, iniciando cada palabra con letra mayúscula. Esta práctica mejora la legibilidad y permite diferenciar una clase de una variable o función.

La clase Libro agrupa datos y comportamientos relacionados. Sus datos son los atributos `titulo`, `autor` e `isbn`. Sus comportamientos son los métodos `mostrar_informacion()` y `__str__()`. Esta agrupación es una característica esencial de la POO, porque evita que la información del libro y las acciones relacionadas con ella queden dispersas en distintas partes del programa.

El estudiante debe comprender que una clase no es todavía un objeto concreto. La clase es la definición. El objeto aparece cuando se crea una instancia de esa clase. Por ejemplo, `libro1` y `libro2` serán objetos concretos creados a partir de `Libro`.

1.4.3 Constructor, self y atributos

Un programador orientado a objetos inicia el análisis desde una pregunta diferente: ¿qué objetos existen dentro del sistema? En lugar de centrarse únicamente en las acciones, identifica las entidades que participan en el problema, sus características y sus responsabilidades.

El constructor es un método especial que se ejecuta automáticamente cuando se crea un objeto. En Python, el constructor se define con el método `__init__()`. Su función principal es inicializar los atributos del objeto, es decir, asignar los valores iniciales que tendrá cada instancia.

En la clase Libro, el constructor recibe tres datos: titulo, autor e isbn. Estos datos se asignan a los atributos del objeto mediante self. La palabra self representa al objeto actual. Gracias a self, cada objeto puede almacenar sus propios valores sin mezclarse con los valores de otros objetos.

```
def __init__(self, titulo, autor, isbn):  
    self.titulo = titulo  
    self.autor = autor  
    self.isbn = isbn
```

La expresión `self.titulo = titulo` significa que el objeto actual tendrá un atributo llamado titulo y que ese atributo recibirá el valor enviado al constructor. Lo mismo ocurre con `self.autor` y `self.isbn`. De esta manera, cada libro creado a partir de la clase Libro tendrá su propio título, su propio autor y su propio ISBN.

Este punto suele generar confusión en los estudiantes, porque los nombres pueden ser iguales en ambos lados de la asignación. Sin embargo, el significado es diferente. El nombre que aparece después de self pertenece al objeto. El nombre que aparece a la derecha corresponde al parámetro recibido por el constructor.

Por ejemplo, cuando se ejecuta `Libro("Python para Principiantes", "Juan Pérez", "ISBN001")`, Python crea un nuevo objeto y llama automáticamente al constructor. El valor "Python para Principiantes" se guarda en `self.titulo`, el valor "Juan Pérez" se guarda en `self.autor` y el valor "ISBN001" se guarda en `self.isbn`.

```
libro1 = Libro(
    "Python para Principiantes",
    "Juan Pérez",
    "ISBN001"
)
```

El constructor permite que los objetos nazcan con información inicial. Sin este método, sería necesario crear el objeto vacío y luego asignar manualmente cada atributo. Esto haría que el código fuera más extenso, menos claro y más propenso a errores.

Elemento	Función dentro del constructor
<code>__init__</code>	Método especial que inicializa el objeto.
<code>self</code>	Referencia al objeto actual.
<code>titulo, autor, isbn</code>	Parámetros que recibe el constructor.
<code>self.titulo, self.autor, self.isbn</code>	Atributos que quedan almacenados dentro del objeto.

Nota. *self no es un dato adicional del libro. Es la referencia que permite acceder al objeto actual y a sus atributos internos.*

1.4.4 Métodos y comportamientos de los objetos

Los métodos son funciones definidas dentro de una clase. Representan acciones, operaciones o comportamientos que un objeto puede realizar. A diferencia de una función externa, un método pertenece a una clase y normalmente trabaja con los atributos del objeto mediante `self`.

En la clase `Libro` se define el método `mostrar_informacion()`. Este método no recibe título ni autor como parámetros externos, porque ya puede acceder a esos datos mediante `self.titulo` y `self.autor`. Esta es una de las ventajas de la POO: los métodos pueden trabajar directamente con la información interna del objeto.

```
def mostrar_informacion(self):  
    return f"Título: {self.titulo} | Autor: {self.autor}"
```

El método devuelve una cadena de texto con información del libro. La palabra `return` indica que el método entrega un resultado. En este caso, el resultado es una representación más detallada del título y del autor.

Los métodos permiten que el comportamiento esté unido a los datos. En lugar de tener una función externa que reciba un diccionario o una lista con información del libro, la propia clase `Libro` contiene el comportamiento necesario para mostrar sus datos. Esto mejora la organización y reduce la dependencia de estructuras externas.

En POO, un buen método debe estar relacionado con la responsabilidad de la clase. Por ejemplo, `mostrar_informacion()` tiene sentido dentro de `Libro` porque muestra información del libro. En cambio, un método para registrar usuarios no debería colocarse dentro de `Libro`, porque esa no es su responsabilidad.

Nota. *Los métodos permiten que los objetos no sean únicamente contenedores de datos, sino entidades capaces de ejecutar comportamientos relacionados con su información.*

1.4.5 Método especial `__str__()`

Python posee métodos especiales que permiten personalizar el comportamiento de los objetos. Estos métodos se reconocen porque comienzan y terminan con doble guion bajo. Uno de los más utilizados en programación orientada a objetos es `__str__()`.

El método `__str__()` define cómo se representará un objeto cuando se convierta a texto o cuando se utilice la función `print()`. Si una clase no define este método, Python puede mostrar una representación poco legible del objeto, generalmente relacionada con su ubicación interna en memoria.

En la clase `Libro`, el método `__str__()` permite mostrar el título y el autor de forma clara.

```
def __str__(self):  
    return f"{self.titulo} - {self.autor}"
```

Cuando se imprime un objeto de tipo Libro, Python llama automáticamente a `__str__()`. Por esta razón, en el archivo biblioteca.py se puede utilizar `print(libro)` dentro de un ciclo y obtener una salida comprensible para el usuario.

La diferencia entre `mostrar_informacion()` y `__str__()` es importante. `mostrar_informacion()` es un método normal que debe llamarse explícitamente. En cambio, `__str__()` se ejecuta automáticamente cuando Python necesita representar el objeto como texto.

Método	Uso principal
<code>mostrar_informacion()</code>	Devuelve información del objeto cuando el programador llama explícitamente al método.
<code>__str__()</code>	Define la representación textual del objeto cuando se utiliza <code>print()</code> o <code>str()</code> .

El uso de `__str__()` es especialmente útil durante el aprendizaje, porque permite observar los objetos de forma clara durante la ejecución del programa. También ayuda en la depuración, ya que facilita verificar qué objetos se han creado y qué información contienen.

Nota. El método `__str__()` no es obligatorio para que una clase funcione, pero mejora la claridad de la salida del programa y la comprensión de los objetos creados.

1.4.6 Organización modular del proyecto

La modularidad es una característica clave en el desarrollo de software. Consiste en dividir un programa en partes independientes, donde cada archivo o módulo cumple una responsabilidad específica. Esta organización evita que todo el código se concentre en un solo archivo y facilita el mantenimiento del proyecto.

En esta semana se utiliza la siguiente estructura:

```
biblioteca_app/  
├─ modelos/  
│   ├── libro.py  
│   └── usuario.py  
├─ servicios/  
│   └── biblioteca.py  
└─ main.py
```

La carpeta modelos contiene las clases que representan entidades principales del sistema. En este caso, Libro y Usuario. La carpeta servicios contiene la lógica que administra esas entidades. En este caso, Biblioteca. El archivo main.py funciona como punto de arranque, es decir, el lugar desde donde se inicia la ejecución del programa.

Esta estructura no debe interpretarse todavía como una arquitectura empresarial compleja. Para este nivel, se utiliza como una forma introductoria de separación de responsabilidades. El objetivo es que el estudiante comprenda que cada parte del programa debe tener una función clara.

Si todo el código estuviera en main.py, el programa podría funcionar, pero sería más difícil de leer y mantener. Al separar modelos, servicios y ejecución, el proyecto se vuelve más ordenado y preparado para crecer en futuras semanas, por ejemplo, cuando se incorpore una interfaz gráfica con Tkinter.

Nota. La organización modular permite que el estudiante comience a pensar como desarrollador: no solo importa que el programa funcione, también importa que esté bien organizado.

1.4.7 Modelos: representación de entidades del sistema

Los modelos representan las entidades principales del sistema. Una entidad es un elemento importante del problema que necesita ser representado mediante datos y comportamientos. En el proyecto de biblioteca, las entidades principales son Libro y Usuario.

El archivo usuario.py contiene la clase Usuario. Esta clase representa a una persona registrada dentro del sistema. Sus atributos son nombre y correo. También contiene un método para mostrar información y un método especial `__str__()` para representar al usuario como texto.

```
# modelos/usuario.py

class Usuario:

    def __init__(self, nombre, correo):
        self.nombre = nombre
        self.correo = correo

    def mostrar_informacion(self):
        return f"{self.nombre} ({self.correo})"

    def __str__(self):
        return self.nombre
```

Esta clase sigue el mismo patrón que Libro. Tiene un constructor, atributos, un método de información y un método `__str__()`. Esta repetición no es negativa; al contrario, ayuda al estudiante a reconocer una estructura común para construir clases.

El modelo Usuario no administra todos los usuarios del sistema. Solo representa un usuario individual. La administración de varios usuarios corresponde a otro componente del proyecto: la clase Biblioteca, ubicada en la carpeta servicios.

Esta separación es fundamental. Si cada clase intenta hacer demasiadas cosas, el código se vuelve confuso. En cambio, cuando cada clase tiene una responsabilidad clara, el sistema resulta más fácil de entender.

Nota. Un modelo debe representar una entidad. No debe convertirse en el lugar donde se coloca toda la lógica del sistema.

1.4.8 Servicios: lógica de operación y administración de objetos

Los servicios contienen operaciones o lógica de administración. En este proyecto, la clase Biblioteca se ubica dentro de la carpeta servicios porque se encarga de administrar objetos Libro y Usuario. Su responsabilidad no es representar un libro individual ni un usuario individual, sino gestionar colecciones de ellos.

```
# servicios/biblioteca.py

class Biblioteca:

    def __init__(self):
        self.libros = []
        self.usuarios = []

    def agregar_libro(self, libro):
        self.libros.append(libro)

    def agregar_usuario(self, usuario):
        self.usuarios.append(usuario)

    def mostrar_libros(self):
        for libro in self.libros:
            print(libro)

    def mostrar_usuarios(self):
        for usuario in self.usuarios:
            print(usuario)
```

El constructor de Biblioteca crea dos listas vacías: libros y usuarios. Estas listas funcionarán como contenedores internos donde se almacenarán los objetos agregados al sistema. En esta etapa todavía no se utiliza una base de datos ni archivos externos, porque el propósito es comprender la relación entre objetos y la organización del código.

El método `agregar_libro(self, libro)` recibe un objeto `Libro` y lo agrega a la lista `self.libros`. El método `agregar_usuario(self, usuario)` realiza una operación equivalente con objetos `Usuario`. Esto demuestra que un objeto puede administrar otros objetos.

Los métodos `mostrar_libros()` y `mostrar_usuarios()` recorren las listas mediante ciclos `for`. Dentro de cada ciclo se utiliza `print()`. Gracias al método `__str__()` definido en `Libro` y `Usuario`, la salida será legible. Si los modelos no tuvieran `__str__()`, la impresión de cada objeto sería menos clara.

La clase `Biblioteca` representa un paso importante en el aprendizaje, porque muestra cómo una clase puede coordinar objetos creados a partir de otras clases. Esta relación es una base para desarrollar sistemas más completos, como inventarios, sistemas académicos, agendas, bibliotecas digitales o aplicaciones con interfaz gráfica.

***Nota.** Un servicio administra operaciones. En este proyecto, `Biblioteca` funciona como componente que gestiona colecciones de libros y usuarios.*

1.4.9 Main.py: punto de arranque de la aplicación

El archivo `main.py` es el punto de arranque del programa. Su función principal es coordinar la ejecución inicial. En este archivo se importan las clases, se crean objetos y se llama a los métodos necesarios para observar el funcionamiento del sistema.

En esta semana no se agrega todavía un menú interactivo, porque el objetivo no es construir un programa completo de consola, sino comprender cómo se conectan los módulos. Posteriormente, esta misma estructura puede evolucionar hacia menús, manejo de archivos, interfaces gráficas o bases de datos.

```

# main.py
from modelos.libro import Libro
from modelos.usuario import Usuario
from servicios.biblioteca import Biblioteca

# Crear objetos Libro
libro1 = Libro(
    "Python para Principiantes",
    "Juan Pérez",
    "ISBN001"
)
libro2 = Libro(
    "Programación Orientada a Objetos",
    "María Gómez",
    "ISBN002"
)

# Crear objeto Usuario
usuario1 = Usuario(
    "Carlos López",
    "carlos@correo.com"
)

# Crear biblioteca
biblioteca = Biblioteca()

# Agregar objetos a la biblioteca
biblioteca.agregar_libro(libro1)
biblioteca.agregar_libro(libro2)

biblioteca.agregar_usuario(usuario1)

print("=== LIBROS REGISTRADOS ===")
biblioteca.mostrar_libros()

print("
=== USUARIOS REGISTRADOS ===")
biblioteca.mostrar_usuarios()

```


La primera parte del archivo contiene importaciones. Estas líneas permiten utilizar clases que fueron definidas en otros archivos. Luego se crean objetos de tipo Libro y Usuario. Cada objeto recibe valores concretos para sus atributos.

Después se crea un objeto Biblioteca. Este objeto será responsable de administrar los libros y usuarios. Posteriormente, los objetos libro1, libro2 y usuario1 se agregan a la biblioteca mediante los métodos agregar_libro() y agregar_usuario(). Finalmente, se muestran los datos registrados.

Este flujo permite observar claramente cómo se comunican los módulos del proyecto. main.py no define la clase Libro ni la clase Usuario. Tampoco contiene directamente la lógica de administración. Su función es coordinar la ejecución utilizando los componentes definidos en otros archivos.

Nota. main.py debe mantenerse como punto de arranque. Si se sobrecarga con demasiada lógica, el proyecto pierde claridad.

1.4.10 Relación entre objetos

Una de las ventajas más importantes de la POO es que los objetos pueden interactuar entre sí. Esta interacción permite representar relaciones del mundo real. En una biblioteca real existen libros, usuarios y procesos de administración. En el proyecto, estas relaciones se representan mediante objetos.

El objeto biblioteca puede recibir objetos libro y usuario. Esto se observa en las siguientes líneas:

```
biblioteca.agregar_libro(libro1)
biblioteca.agregar_libro(libro2)
biblioteca.agregar_usuario(usuario1)
```

Cada línea indica una interacción entre objetos. biblioteca es un objeto de la clase Biblioteca. libro1 y libro2 son objetos de la clase Libro. usuario1 es un objeto de la clase Usuario. Cuando se llama a agregar_libro(), la biblioteca recibe un libro y lo almacena en su lista interna.

Esta relación permite comprender que los objetos no existen aislados. En un sistema orientado a objetos, cada objeto cumple una responsabilidad, pero también puede colaborar con otros objetos para alcanzar un resultado.

Esta idea es esencial para desarrollar sistemas de complejidad media. Un sistema académico, por ejemplo, puede tener objetos Estudiante, Docente, Materia y Calificación. Cada uno tiene datos propios, pero deben relacionarse para construir procesos más completos.

Nota. La POO permite representar entidades y también las relaciones que existen entre ellas dentro de un sistema.

1.4.11 Uso de importaciones en Python

Las importaciones permiten utilizar código definido en otros archivos. En proyectos pequeños, todo el código puede escribirse en un solo archivo. Sin embargo, cuando se trabaja con modularidad, cada clase se ubica en un archivo diferente y se necesita importar lo que se va a utilizar.

En main.py se observan las siguientes importaciones:

```
from modelos.libro import Libro
from modelos.usuario import Usuario
from servicios.biblioteca import Biblioteca
```

La primera línea indica que desde el módulo modelos.libro se importa la clase Libro. Esto significa que Python buscará dentro de la carpeta modelos el archivo libro.py y, dentro de

ese archivo, la clase Libro. La segunda línea realiza un proceso similar con Usuario. La tercera línea importa Biblioteca desde la carpeta servicios.

Este mecanismo evita duplicar código. Si la clase Libro ya fue definida en libro.py, no es necesario volver a escribirla en main.py. Basta con importarla. Esta práctica es fundamental en el desarrollo de software porque favorece la reutilización y la separación de responsabilidades.

Para que las importaciones funcionen correctamente, la estructura de carpetas debe estar bien organizada y el programa debe ejecutarse desde el nivel adecuado del proyecto. En este caso, main.py se encuentra al mismo nivel que las carpetas modelos y servicios.

Importación	Significado
from modelos.libro import Libro	Permite utilizar la clase Libro definida en modelos/libro.py.
from modelos.usuario import Usuario	Permite utilizar la clase Usuario definida en modelos/usuario.py.
from servicios.biblioteca import Biblioteca	Permite utilizar la clase Biblioteca definida en servicios/biblioteca.py.

Nota. Las importaciones son el puente que permite que los módulos de un proyecto Python se comuniquen entre sí.

1.4.12 Flujo general de ejecución del proyecto

El flujo de ejecución indica el orden general en que se utiliza el proyecto. En este ejemplo, el programa inicia en main.py. Desde allí se importan las clases, se crean objetos y se ejecutan métodos.

```
main.py
|
|— importa Libro desde modelos/libro.py
|— importa Usuario desde modelos/usuario.py
|— importa Biblioteca desde servicios/biblioteca.py
```

Luego:

1. Crea objetos Libro.
2. Crea un objeto Usuario.

3. Crea un objeto Biblioteca.
4. Agrega libros y usuarios a la biblioteca.
5. Muestra la información registrada.

Este flujo demuestra que el programa no depende de un solo bloque de código. Cada componente participa desde su responsabilidad. Los modelos definen entidades, el servicio administra colecciones y main.py coordina la ejecución.

La salida esperada del programa será una lista de libros registrados y una lista de usuarios registrados. La forma en que se muestran estos objetos depende directamente de los métodos `__str__()` definidos en las clases Libro y Usuario.

```
=== LIBROS REGISTRADOS ===  
Python para Principiantes - Juan Pérez  
Programación Orientada a Objetos - María Gómez  
  
=== USUARIOS REGISTRADOS ===  
Carlos López
```

La salida confirma que los objetos fueron creados correctamente, agregados al servicio Biblioteca y mostrados mediante los métodos correspondientes. Aunque el programa es sencillo, contiene elementos esenciales para avanzar hacia aplicaciones más completas.

Nota. Comprender el flujo de ejecución ayuda a identificar dónde se crea cada objeto, qué módulo lo administra y qué archivo inicia el programa.

1.4.13 Beneficios de la organización modular

La organización modular ofrece ventajas importantes en el desarrollo de software. Aunque al inicio puede parecer más sencillo escribir todo en un solo archivo, esta práctica se vuelve limitada cuando el proyecto crece. La modularidad permite distribuir el código en partes más pequeñas y comprensibles.

Uno de los principales beneficios es la claridad. Cuando los modelos se ubican en una carpeta, los servicios en otra y la ejecución en main.py, resulta más fácil encontrar cada parte del programa. También mejora el mantenimiento, porque un cambio en una clase puede realizarse en su archivo correspondiente sin modificar todo el proyecto.

La reutilización es otro beneficio importante. Una clase definida en modelos puede ser utilizada por diferentes servicios o interfaces. Por ejemplo, la clase Libro podría reutilizarse más adelante en una interfaz gráfica con Tkinter sin necesidad de modificar su definición básica.

La escalabilidad también se ve favorecida. Si en el futuro el sistema necesita registrar préstamos, devoluciones o reportes, se pueden agregar nuevas clases o servicios sin desorganizar el código existente.

Código poco organizado	Código modular
Todo se escribe en un solo archivo.	El proyecto se divide en carpetas y módulos.
Es difícil localizar responsabilidades.	Cada archivo cumple una función específica.
Aumenta la repetición de código.	Se favorece la reutilización de clases.
El mantenimiento se vuelve complejo.	Los cambios se realizan con mayor claridad.
El crecimiento puede desordenar el programa.	El sistema puede ampliarse de forma progresiva.

Nota. *La modularidad no hace que el programa sea automáticamente perfecto, pero ofrece una base más ordenada para crecer y mantenerse.*

1.5. CONCLUSIONES

La Programación Orientada a Objetos facilita la representación de problemas reales mediante clases y objetos, permitiendo modelar entidades del sistema de forma más comprensible.

Los atributos y métodos permiten encapsular información y comportamientos dentro de una misma entidad, evitando que los datos y las acciones queden dispersos en el programa.

La modularidad contribuye a organizar el software en componentes independientes y fáciles de mantener, como modelos, servicios y un archivo principal de ejecución.

Una correcta organización del proyecto favorece el desarrollo de aplicaciones escalables, reutilizables y de mayor calidad, preparando al estudiante para proyectos más completos en Python.

1.6. BIBLIOGRAFÍA

Boucheny, V. (2025). Aprender programación orientada a objetos con Python (3.^a ed.). Ediciones ENI.

Downey, A. (2024). Think Python (3rd ed.). O'Reilly Media.

Hernández Bejarano, M., & Baquero Rey, L. E. (2023). Programación orientada a objetos en Java: Buenas prácticas. Ediciones de la U.

Hernández Bejarano, M., & Baquero Rey, L. E. (2025). Python con orientación a objetos y al análisis de datos. Ediciones de la U.

Piñeiro Gómez, J. M. (2022). Principios de la programación orientada a objetos. Paraninfo.